

PalmSource®

Conference and Expo



Debugging with PalmDebugger

Doug Anderson, Software Engineer for Handspring



PalmDebugger is

- A powerful assembly-level debugger for Palm OS
- A source-level debugger when using prc-tools
- A difficult tool to get started with
- Available free in Palm's "Extra Tools"

This session will get you started with PalmDebugger's assembly-level debugger and show how to use it to debug a crash



Agenda

- Introduction to PalmDebugger
- Basic Debugger Commands
- Debugging a Crash
- More Advanced Techniques
- Q & A



When to Use the Assembly-Level Debugger

- You may not be able to get user / tester device into the source level debugger
- The crash may happen in someone else's code, like an OS routine or shared library
- If you're developing shared libraries, you may not have source-level debugging
- It can help you understand what the compiler is doing

Connecting to PalmDebugger (part 1)

1. Close or disable HotSync® Manager
2. Run the PalmDebugger application
3. Choose the proper connection from the “Connection” menu; if serial, make sure you’ve chosen 57600 baud
4. Connect your device to the proper cable/cradle

Connecting to PalmDebugger (part 2)

5. In a text field on your device, enter:
   <shortcut> . 1
   …which is a cursive, lowercase ‘el’, a period, and the number 1
6. Verify that a message similar to the following appears in the “Debugger” window of PalmDebugger:

```
EXCEPTION ID = $A0
  'PrvHandleEvent*'
  +$087A  1000D608  *MOVEQ.L   #$01,D0
```

Troubleshooting Your Connection

- Making the initial connection can be tricky
- Unplugging / replugging your cable can help; also try typing “att” in Debugger window
- Some OEMs only support debugging over certain communication methods—serial is most common
- If all else fails, consult developer documentation for your device, or come to open labs



Where Are We Now?

- Device is “stopped” in the debugger; no code is executing
- Device is drawing a whole lot of power, so don’t leave it in the debugger for too long
- We can now inspect the device, or step through as instructions are executed



Agenda

- Introduction to PalmDebugger
- Basic Debugger Commands
- Debugging a Crash
- More Advanced Techniques
- Q & A

Entering Debugger Commands

- On Windows: type a command in the “Debugger” window and hit Enter
- On Macintosh: type a command in the “Debugger” window and hit Command-Return, or the Enter key on the keypad
- Think of the debugger window as a scratchpad—clicking on any line and hitting enter on a line executes that line

The Instruction List Command

- Syntax: `il [<addr> [<count>]]`
- Disassembles machine instructions at the given address
- When stopped in the debugger, you can use this to figure out what just happened and what will happen next
- If no address is specified, disassembles starting with the next instruction that will be executed



The Display Memory Command

- Syntax: `dm <addr> [<count>]`
- You can use this to look at the values stored in any memory location

The Stack Crawl Command

- Syntax: `sc [<addr> [<frames>]]`
- This command shows you the current call chain (all the functions that called the current one)
- Assumes that you have “A6 Frame Pointers” enabled in your compiler, as well as “MacsBug labels”; luckily, the OS is compiled with these options
- Normally, just use “`sc`” with no parameters, but you can get more with “`sc A6 20`”



Flow Control Command: step over

- Syntax: `t`
- This is like the step command in most source-level debuggers—it steps one instruction
- Function calls and system traps are treated as a single instruction



Flow Control Command: step into

- Syntax: `s`
- This is like the step command in most source-level debuggers—it steps one instruction
- Function calls and system traps are stepped into



Flow Control Command: go

- Syntax: `g`
- This command leaves the debugger and allows the device to resume running normally



The help command

- Syntax: **`help`**
- There are many more commands than we can cover here; help lists them
- Help command also gives syntax for commands



Agenda

- Introduction to PalmDebugger
- Basic Debugger Commands
- Debugging a Crash
- More Advanced Techniques
- Q & A

Getting to the Debugger When you Need It

- Using “shortcut . 1” doesn’t let you debug your own code
- PalmOS will drop you into the debugger instead of giving a fatal alert if you’ve already entered the debugger at least once since the last reset
- You can enter the debugger the first time with “shortcut . 1”, or fool PalmOS® with a tool like Handspring’s DebugPrefs (available on Handspring developer web site)

Getting to the Debugger (Part 2)

- You can very easily force a PalmOS fatal error by using `DbgBreak()` in your C code
- On the Emulator, you can drop into the debugger any time by typing "`att`" in PalmDebugger's "Debugger" window, which is handy for stopping in infinite loops
- In newer Handspring devices (OS 3.5+), you can sometimes get into the debugger by pressing the HotSync button in a Fatal Alert dialog



Debugging a Fatal Error

- Scenario: a user comes to you with a Fatal Error that happens on their device
- Goal: find the source of the Fatal Error
- Assume: somehow you can get to the crashed device to enter the debugger using the already discussed techniques



Step 1: The stack crawl

- Use "`sc A6 20`" to get an idea of where you are
- Look at the right side of the stack crawl for routine names
- This will often point out the problem right away, since you know what functions were called and you can scrutinize that part of your code

Step 2: Disassemble

- Use "`il <addr> <count>`" to figure out what part of routines you are in
- Looking back at your stack crawl, the "Caller" column gives the address that will be returned to
- You can disassemble around an address by passing parameters to "`il`" for addr and count:

```
il <addr>-10 20
```

  Note: there's no space around the '-'
- Use system traps to match up assembly code to C (don't need to understand assembly!)

Step 3: Look at Function Parameters

- Use "`dm <addr>`" look at memory to figure out what the parameters to a function were
- First parameter (left to right) is stored 8 bytes past the start of an A6 Frame. See it with:

```
dm <A6 Frame>+08
```

- Second parameter is stored right afterward. So if first is 4 bytes, the second would be stored 12 bytes past the A6 Frame. Note that PalmDebugger uses Hexadecimal:

```
dm <A6 Frame>+0C
```



Agenda

- Introduction to PalmDebugger
- Basic Debugger Commands
- Debugging a Crash
- More Advanced Techniques
- Q & A



Using Debug Messages

- PalmDebugger can also act as an output window for debug messages
- Use "`DbgMessage(string)`" in your C code to send output to the debugger
- Remember to take out debug messages before shipping, as they can slow down or freeze a user's device.

Down-Arrow Reset to Debug Boot Code

- Assembly-level debugging is often useful during boot code (reset launch codes, serial drivers, shared libraries, etc)
- How do you enter the debugger “at least once”?
- Reset the device while in the cradle with the down-arrow pressed
- When debugger enters, hold down arrow again and <g>o in the debugger
- When debugger enters again, <g>o again
- Now your code will enter the debugger



For More Info

- A more complete tutorial on PalmDebugger will hopefully show up on Handspring's developer web site soon
- Come to the open labs and ask questions
- Come to the PalmDebugger lab



Agenda

- Introduction to PalmDebugger
- Basic Debugger Commands
- Debugging a Crash
- More Advanced Techniques
- Q & A



PalmSource®

Conference and Expo